COMPETITION MASTER 03 — Wiren Board 7.4 Complete Smart Farm

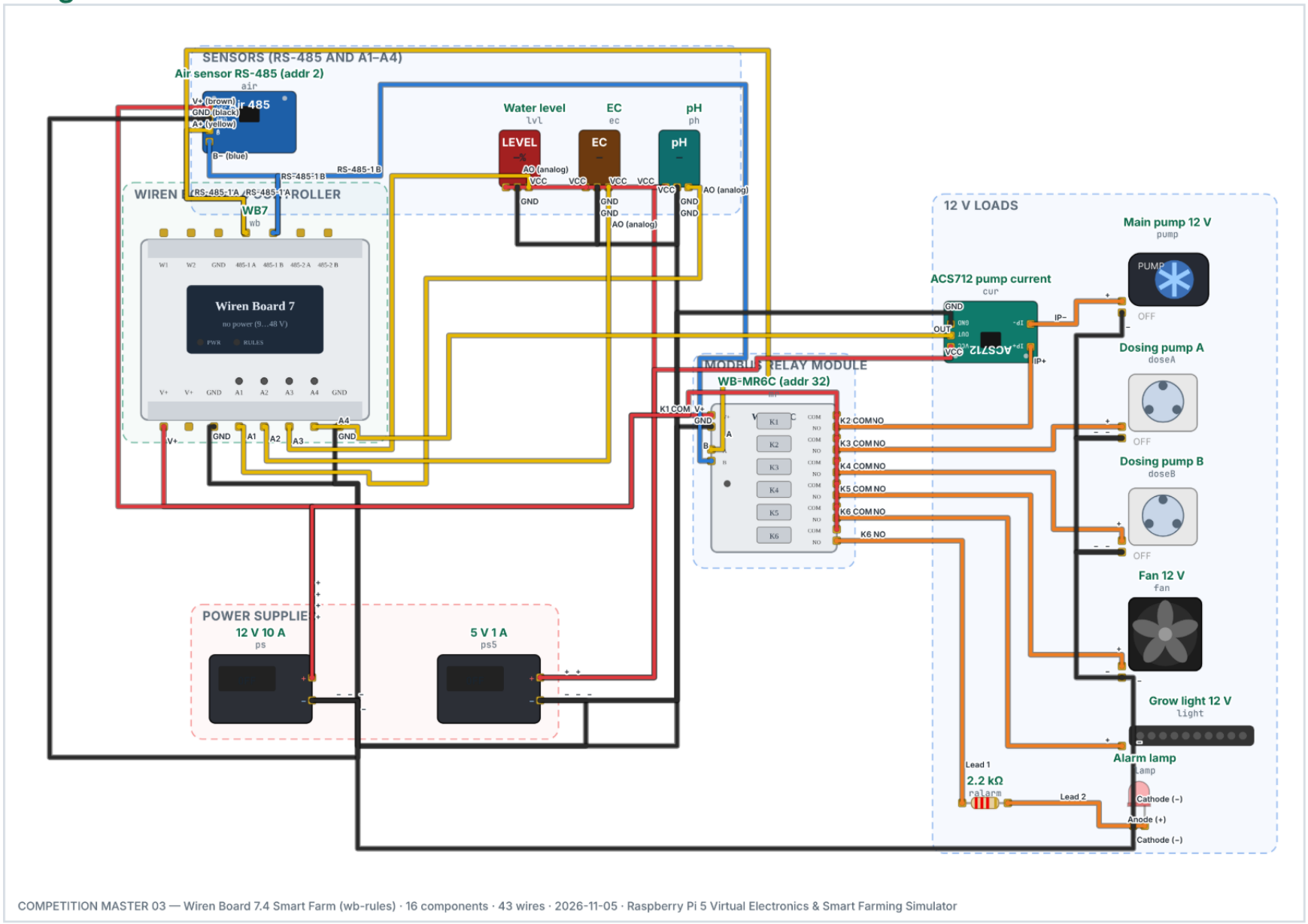
Complete study file

The same farm built the Wiren Board way: Modbus RTU devices on RS-485, the ADC inputs A1–A4, a WB-MR6C relay module, and rules that read and write MQTT controls.

Controller	Wiren Board 7.4
Language	wb-rules (JavaScript ES5) — defineRule, defineVirtualDevice, dev[], setTimeout
Program file	wb/rules.js
Status display	MQTT tab: the virtual device "farm" (values + DOSING/TEST pushbuttons) and the wb-rules log (STATUS lines).
Parts / wires	16 / 43
Exported	2026-11-05

RS-485-1: air sensor (temperature, humidity, CO_2, address 2) + WB-MR6C (address 32: K1 pump, K2 dose A, K3 dose B, K4 fan, K5 light, K6 alarm). A1 pH, A2 EC, A3 level, A4 ACS712. 12 V and 5 V supplies.

1. Circuit diagram



COMPETITION MASTER 03 — Wiren Board 7.4 Smart Farm (wb-rules) · 16 components · 43 wires · 2026-11-05 · Raspberry Pi 5 Virtual Electronics & Smart Farming Simulator

16 parts · 43 wires

2. Wiring table — every wire explained

#	From	To	Type	Why	If disconnected
1	12 V 10 A · +	WB7 · V+	Power	power for the Wiren Board 7.4	the Wiren Board 7.4 loses power — the Wiren Board loses power: the rules, MQTT and Modbus stop
2	12 V 10 A · –	WB7 · GND	Ground	ground (return path) of the Wiren Board 7.4	the Wiren Board 7.4 loses power — the Wiren Board loses power: the rules, MQTT and Modbus stop
3	5 V 1 A · –	WB7 · GND	Ground	common ground: the 5 V parts and the controller must share GND so their signals have the same reference	the parts on this wire lose 5 V; without the common GND wire every signal loses its reference and the readings are wrong
4	12 V 10 A · +	Air sensor RS-485 (addr 2) · V+ (brown)	Power	power for the RS-485 air sensor (address 2)	the RS-485 air sensor (address 2) loses power — Modbus timeouts: Temperature/Humidity/CO2 get an error, the reading is lost: SENSOR ERROR alarm, alarm lamp on, fan forced ON
5	Air sensor RS-485 (addr 2) · GND (black)	12 V 10 A · –	Ground	ground (return path) of the RS-485 air sensor (address 2)	the RS-485 air sensor (address 2) loses power — Modbus timeouts: Temperature/Humidity/CO2 get an error, the reading is lost: SENSOR ERROR alarm, alarm lamp on, fan forced ON
6	WB7 · RS-485-1 A	Air sensor RS-485 (addr 2) · A+ (yellow)	Communication bus	RS-485 air sensor (address 2): RS-485 bus (Modbus RTU, A↔A, B↔B)	Modbus timeouts: Temperature/Humidity/CO2 get an error, the reading is lost: SENSOR ERROR alarm, alarm lamp on, fan forced ON
7	WB7 · RS-485-1 B	Air sensor RS-485 (addr 2) · B− (blue)	Communication bus	RS-485 air sensor (address 2): RS-485 bus (Modbus RTU, A↔A, B↔B)	Modbus timeouts: Temperature/Humidity/CO2 get an error, the reading is lost: SENSOR ERROR alarm, alarm lamp on, fan forced ON
8	12 V 10 A · +	WB-MR6C (addr 32) · V+	Power	power for the WB-MR6C relay module (address 32)	the WB-MR6C relay module (address 32) loses power — the relay module stops answering: no load can be switched, wb-mqtt-serial reports errors
9	WB-MR6C (addr 32) · GND	12 V 10 A · –	Ground	ground (return path) of the WB-MR6C relay module (address 32)	the WB-MR6C relay module (address 32) loses power — the relay module stops answering: no load can be switched, wb-mqtt-serial reports errors
10	WB7 · RS-485-1 A	WB-MR6C (addr 32) · A	Communication bus	WB-MR6C relay module (address 32): RS-485 bus (Modbus RTU): the relays are switched by Modbus commands	the relay module stops answering: no load can be switched, wb-mqtt-serial reports errors
11	WB7 · RS-485-1 B	WB-MR6C (addr 32) · B	Communication bus	WB-MR6C relay module (address 32): RS-485 bus (Modbus RTU): the relays are switched by Modbus commands	the relay module stops answering: no load can be switched, wb-mqtt-serial reports errors
12	5 V 1 A · +	pH · VCC	Power	power for the pH sensor module	the pH sensor module loses power — pH reads wrong (near 0 → below 1): SENSOR ERROR
13	pH · GND	5 V 1 A · –	Ground	ground (return path) of the pH sensor module	the pH sensor module loses power — pH reads wrong (near 0 → below 1): SENSOR ERROR
14	pH · VCC	EC · VCC	Power	power for the pH sensor module	the pH sensor module loses power — pH reads wrong (near 0 → below 1): SENSOR ERROR
15	EC · GND	pH · GND	Ground	ground (return path) of the pH sensor module	the pH sensor module loses power — pH reads wrong (near 0 → below 1): SENSOR ERROR
16	EC · VCC	Water level · VCC	Power	power for the EC sensor module	the EC sensor module loses power — EC reads wrong (near 0): OUT OF RANGE
17	Water level · GND	EC · GND	Ground	ground (return path) of the EC sensor module	the EC sensor module loses power — EC reads wrong (near 0): OUT OF RANGE
18	pH · AO (analog)	WB7 · A1	Analog signal	pH sensor module: analog voltage: pH = 3.5 × V	pH reads wrong (near 0 → below 1): SENSOR ERROR
19	EC · AO (analog)	WB7 · A2	Analog signal	EC sensor module: analog voltage: EC = V ÷ 0.5	EC reads wrong (near 0): OUT OF RANGE

#	From	To	Type	Why	If disconnected
20	Water level · AO (analog)	WB7 · A3	Analog signal	water-level sensor: analog voltage: 3 V = 100 %	level reads wrong (near 0 %): LOW WATER, irrigation and dosing refused
21	5 V 1 A · +	ACS712 pump current · VCC	Power	power for the ACS712 current sensor	the ACS712 current sensor loses power — the pump current reads wrong: the next irrigation reports PUMP FAILURE
22	ACS712 pump current · GND	5 V 1 A · –	Ground	ground (return path) of the ACS712 current sensor	the ACS712 current sensor loses power — the pump current reads wrong: the next irrigation reports PUMP FAILURE
23	ACS712 pump current · OUT	WB7 · A4	Analog signal	ACS712 current sensor: analog voltage: 2.5 V + 0.185 V per ampere of pump current	the pump current reads wrong: the next irrigation reports PUMP FAILURE
24	ACS712 pump current · IP–	Main pump 12 V · +	Load circuit (12 V)	12 V circuit of the main pump: supply + → relay contact → load → supply –	the pump cannot run: no current through the ACS712 → PUMP FAILURE: no current
25	2.2 kΩ · Lead 2	Alarm lamp · Anode (+)	Digital data	alarm lamp: 12 V lamp (LED + 2.2 kΩ)	alarms are still printed but the lamp never lights
26	12 V 10 A · +	WB-MR6C (addr 32) · K1 COM	Power	power for the main pump	the main pump loses power — the pump cannot run: no current through the ACS712 → PUMP FAILURE: no current
27	WB-MR6C (addr 32) · K1 NO	ACS712 pump current · IP+	Load circuit (12 V)	12 V circuit of the main pump: supply + → relay contact → load → supply –	the pump cannot run: no current through the ACS712 → PUMP FAILURE: no current
28	WB-MR6C (addr 32) · K1 COM	WB-MR6C (addr 32) · K2 COM	Load circuit (12 V)	12 V circuit of the dosing pump A: supply + → relay contact → load → supply –	dose A is not delivered although DOSING A is printed (the Smart Farm totals stay at 0 ml)
29	WB-MR6C (addr 32) · K2 NO	Dosing pump A · +	Load circuit (12 V)	12 V circuit of the dosing pump A: supply + → relay contact → load → supply –	dose A is not delivered although DOSING A is printed (the Smart Farm totals stay at 0 ml)
30	WB-MR6C (addr 32) · K2 COM	WB-MR6C (addr 32) · K3 COM	Load circuit (12 V)	12 V circuit of the dosing pump B: supply + → relay contact → load → supply –	dose B is not delivered although DOSING B is printed
31	WB-MR6C (addr 32) · K3 NO	Dosing pump B · +	Load circuit (12 V)	12 V circuit of the dosing pump B: supply + → relay contact → load → supply –	dose B is not delivered although DOSING B is printed
32	WB-MR6C (addr 32) · K3 COM	WB-MR6C (addr 32) · K4 COM	Load circuit (12 V)	12 V circuit of the fan: supply + → relay contact → load → supply –	the fan never runs: the farm air keeps heating, and the program cannot know it (no fan feedback)
33	WB-MR6C (addr 32) · K4 NO	Fan 12 V · +	Load circuit (12 V)	12 V circuit of the fan: supply + → relay contact → load → supply –	the fan never runs: the farm air keeps heating, and the program cannot know it (no fan feedback)
34	WB-MR6C (addr 32) · K4 COM	WB-MR6C (addr 32) · K5 COM	Load circuit (12 V)	12 V circuit of the grow light: supply + → relay contact → load → supply –	the light stays off although the program prints light=ON
35	WB-MR6C (addr 32) · K5 NO	Grow light 12 V · +	Load circuit (12 V)	12 V circuit of the grow light: supply + → relay contact → load → supply –	the light stays off although the program prints light=ON
36	WB-MR6C (addr 32) · K5 COM	WB-MR6C (addr 32) · K6 COM	Load circuit (12 V)	12 V circuit of the alarm lamp: supply + → relay contact → load → supply –	alarms are still printed but the lamp never lights
37	WB-MR6C (addr 32) · K6 NO	2.2 kΩ · Lead 1	Load circuit (12 V)	12 V circuit of the alarm lamp: supply + → relay contact → load → supply –	alarms are still printed but the lamp never lights
38	Main pump 12 V · –	Dosing pump A · –	Load circuit (12 V)	12 V circuit of the main pump: supply + → relay contact → load → supply –	the pump cannot run: no current through the ACS712 → PUMP FAILURE: no current
39	Dosing pump A · –	Dosing pump B · –	Load circuit (12 V)	12 V circuit of the dosing pump A: supply + → relay contact → load → supply –	dose A is not delivered although DOSING A is printed (the Smart Farm totals stay at 0 ml)
40	Dosing pump B · –	Fan 12 V · –	Load circuit (12 V)	12 V circuit of the fan: supply + → relay contact → load → supply –	the fan never runs: the farm air keeps heating, and the program cannot know it (no fan feedback)
41	Fan 12 V · –	Grow light 12 V · –	Load circuit (12 V)	12 V circuit of the fan: supply + → relay contact → load → supply –	the fan never runs: the farm air keeps heating, and the program cannot know it (no fan feedback)

#	From	To	Type	Why	If disconnected
42	Grow light 12 V · –	Alarm lamp · Cathode (–)	Load circuit (12 V)	12 V circuit of the grow light: supply + → relay contact → load → supply –	the light stays off although the program prints light=ON
43	Alarm lamp · Cathode (–)	12 V 10 A · –	Ground	ground (return path) of the alarm lamp	the alarm lamp loses power — alarms are still printed but the lamp never lights

3. Parts list

#	Component	Model / reference	Qty	Key properties	Voltage	Simulation level	IDs	Assembly notes
1	Wiren Board 7.4 controller	Wiren Board 7.4 · Wiren Board	1	Wi-Fi access point SSID: WirenBoard-AP; Wi-Fi password: wirenboard; IP address (MQTT broker on port 1883): 192.168.42.1; RS-485-1 baud rate: 9600; RS-485-2 baud rate: 9600	9…48 V DC on V+; A1–A4 30 V / 1 A; W1/W2 5 V	Functional Model	wb	
2	DC power supply	Bench / adapter DC supply	1	Voltage: 12 V; Current limit: 10 A	—	Fully Simulated	ps	
3	DC power supply	Bench / adapter DC supply	1	Voltage: 5 V; Current limit: 1 A	—	Fully Simulated	ps5	
4	RS-485 air sensor (temperature, humidity, CO_2 — Modbus RTU)	RS-485 temperature/humidity/CO_2 transmitter	1	Modbus address: 2; Baud rate: 9600	12…24 V DC	Functional Model	air	
5	WB-MR6C Modbus relay module (6 relays)	WB-MR6C v2 · Wiren Board	1	Modbus address: 32; Baud rate: 9600	9…28 V DC	Functional Model	mr	
6	pH sensor (analog)	Analog pH probe + board (SEN0161-style)	1	—	3.3–5 V	Functional Model	ph	
7	EC conductivity sensor (analog)	Analog EC probe + board	1	—	3.3–5 V	Functional Model	ec	
8	Water level sensor (analog)	Analog water-level sensor	1	—	3.3–5 V	Functional Model	lvl	
9	ACS712 current sensor (5 A)	ACS712ELCTR-05B · Allegro	1	Version: 5 A (185 mV/A)	5 V supply	Fully Simulated	cur	
10	Water pump (12 V DC)	12 V DC pump	1	Rated power: 6 W	12 V	Functional Model	pump	
11	Peristaltic dosing pump	12 V peristaltic pump	2	Rated power: 3.6 W	12 V	Functional Model	doseA, doseB	
12	Cooling fan (DC)	12 V DC fan	1	Rated power: 2.4 W	12 V	Functional Model	fan	
13	LED grow light	12 V LED grow bar	1	Rated power: 36 W	12 V	Functional Model	light	
14	Resistor	¼ W carbon-film resistor	1	Resistance: 2.2 kΩ	—	Fully Simulated	ralarm	
15	LED	5 mm LED	1	Colour: red	Vf ≈ 1.9–2.9 V, 20 mA max	Fully Simulated	lamp	
16	Jumper wires	—	43	13× red, 14× black, 6× yellow, 2× blue, 8× orange	—	—		Use male–female jumpers for the Pi header; keep power wires short.

4. Settings (constants at the top of the program)

Name	Value	Meaning
MAX_TEMP	30	فوق 30 درجة: تشغيل المروحة
FAN_OFF_TEMP	27	تحت 27 درجة: إيقاف المروحة
MIN_WATER	20	أقل مستوى ماء مسموح (%) لتشغيل المضخة
PUMP_SECONDS	5	مدة الري بالثواني
IRRIGATION_HOURS	[7, 12, 17]	ساعات الري في اليوم
LIGHT_ON_HOUR	6	ساعة تشغيل الإضاءة
LIGHT_OFF_HOUR	20	ساعة إطفاء الإضاءة
DOSE_A_ML	10	جرعة السماد A بالمليلتر
DOSE_B_ML	5	جرعة السماد B بالمليلتر
FLOW_ML_PER_SEC	2	تدفق مضخة الجرعات (يحتاج معايرة على الجهاز الحقيقي)
PH_MIN	5.5	أقل pH مقبول
PH_MAX	6.5	أعلى pH مقبول
EC_MIN	1.0	أقل EC مقبول (mS/cm)
EC_MAX	2.5	أعلى EC مقبول
MIN_PUMP_AMPS	0.2	أقل تيار يثبت أن المضخة تعمل فعلاً (أمبير)
TEST_SECONDS	2	مدة تشغيل كل جهاز في وضع الاختبار

Training values you may change; they are not certified settings for every crop. Dose accuracy depends on the calibration of the real pumps.

5. The program in 9 sections

1 · Setup and wiring

On the Wiren Board a "pin" is a control name "device/control": the Modbus air sensor (address 2), the four ADC inputs A1–A4 and the six relays of the WB-MR6C (address 32).

- Change a threshold here and the whole program follows it.
- The address in the name (air-sensor_2, wb-mr6c_32) must match the address set in the device.

wb/rules.js

```
// ---------- 1) الإعدادات: نفس القيم في المشاريع الثلاثة ----------
var MAX_TEMP = 30;            // فوق 30 درجة: تشغيل المروحة
var FAN_OFF_TEMP = 27;        // تحت 27 درجة: إيقاف المروحة
var MIN_WATER = 20;           // أقل مستوى ماء مسموح (%) لتشغيل المضخة
var PUMP_SECONDS = 5;         // مدة الري بالثواني
var IRRIGATION_HOURS = [7, 12, 17]; // ساعات الري في اليوم
var LIGHT_ON_HOUR = 6;        // ساعة تشغيل الإضاءة
var LIGHT_OFF_HOUR = 20;      // ساعة إطفاء الإضاءة
var DOSE_A_ML = 10;           // جرعة السماد A بالمليلتر
var DOSE_B_ML = 5;            // جرعة السماد B بالمليلتر
var FLOW_ML_PER_SEC = 2;      // تدفق مضخة الجرعات (يحتاج معايرة على الجهاز الحقيقي)
var PH_MIN = 5.5;             // أقل pH مقبول
var PH_MAX = 6.5;             // أعلى pH مقبول
var EC_MIN = 1.0;             // أقل EC مقبول (mS/cm)
var EC_MAX = 2.5;             // أعلى EC مقبول
var CO2_MAX = 1500;           // أعلى CO2 مقبول (ppm)
var MIN_PUMP_AMPS = 0.2;      // أقل تيار يثبت أن المضخة تعمل فعلاً (أمبير)
var TEST_SECONDS = 2;         // مدة تشغيل كل جهاز في وضع الاختبار
// >>> PINS
// ---------- 2) التوصيلات = أسماء الـControls بصيغة "جهاز/عنصر" ----------
var TEMP = "air-sensor_2/Temperature";   // حساس الهواء RS-485 (عنوان 2): الحرارة
var HUM = "air-sensor_2/Humidity";       // نفس الحساس: الرطوبة
var CO2 = "air-sensor_2/CO2";            // نفس الحساس: CO2
var PH_VOLTS = "wb-adc/A1";              // جهد حساس pH على المدخل A1
var EC_VOLTS = "wb-adc/A2";              // جهد حساس EC على A2
```

```
A3 جهد حساس المستوى على //           ;"var LEVEL_VOLTS = "wb-adc/A3  30
A4 على ACS712 جهد حساس التيار //         ;"var CURRENT_VOLTS = "wb-adc/A4  31
مضخة الري K1 :(عنوان 32) WB-MR6C وحدة المرحّلات //              ;"var PUMP = "wb-mr6c_32/K1  32
A مضخة الجرعات var DOSE_A = "wb-mr6c_32/K2";            // K2  33
B مضخة الجرعات var DOSE_B = "wb-mr6c_32/K3";            // K3  34
المروحة var FAN = "wb-mr6c_32/K4";               // K4  35
الإضاءة var LIGHT = "wb-mr6c_32/K5";             // K5  36
مصباح الإنذار var ALARM_LAMP = "wb-mr6c_32/K6";        // K6  37
 38  // <<< PINS
```

2 · Reading the sensors

One function reads everything and stores it in variables. A missing reading becomes None/null, so the alarms can see it. Analog sensors give volts; one formula per sensor turns volts into pH, EC, % and amperes.

- pH = 3.5 × V · EC = V ÷ 0.5 · level = V ÷ 3 × 100 · amps = (V − 2.5) ÷ 0.185 (the simulator modules; calibrate real probes).
- A Modbus sensor that stops answering gets an error flag: getError() is not empty.

wb/rules.js

```
إذا لم يرد null أو ،Modbus قيمة حساس //              } function sensorValue(name)  80
يضع خطأ عند انقطاع الحساس if (getControl(name).getError() !== "") return null;   // wb-mqtt-serial    81
MQTT القيمة الأخيرة المنشورة عبر //                       ;return dev[name]    82
 83  }
 89  // >>> SENSORS
قراءة كل الحساسات //                  } ()function readSensors  90
(null أو) C° الحرارة //         ;temperature = sensorValue(TEMP)    91
(null أو) % الرطوبة //            ;humidity = sensorValue(HUM)    92
(null أو) co2 = sensorValue(CO2);                 // CO2 ppm    93
الجهد × Python: pH = 3.5 نفس معادلات //         ;ph = round2(dev[PH_VOLTS] * 3.5)    94
0.5 ÷ الجهد = ec = round2(dev[EC_VOLTS] / 0.5);         // EC    95
%100 = فولت waterLevel = Math.round(dev[LEVEL_VOLTS] / 3.0 * 100);   // 3    96
صفر أمبير = فولت pumpAmps = round2((dev[CURRENT_VOLTS] - 2.5) / 0.185);   // ACS712: 2.5    97
 98  }
 99  // <<< SENSORS
```

3 · Driving the outputs

Writing true to "wb-mr6c_32/K1" makes wb-mqtt-serial send a Modbus command to relay K1. setRelay writes only when the state changes, so the bus is not flooded.

- Loads: pump, fan, light and dosing pumps on the 12 V supply, never on the controller pins.
- The alarm lamp is a 12 V lamp on its own relay channel.

wb/rules.js

```
مضخة الري K1 :(عنوان 32) WB-MR6C وحدة المرحّلات //              ;"var PUMP = "wb-mr6c_32/K1  32
A مضخة الجرعات var DOSE_A = "wb-mr6c_32/K2";            // K2  33
B مضخة الجرعات var DOSE_B = "wb-mr6c_32/K3";            // K3  34
المروحة var FAN = "wb-mr6c_32/K4";               // K4  35
الإضاءة var LIGHT = "wb-mr6c_32/K5";             // K5  36
مصباح الإنذار var ALARM_LAMP = "wb-mr6c_32/K6";        // K6  37
تشغيل أو إيقاف مرحّل //             } function setRelay(name, on)  85
(أمر Modbus واحد) نكتب فقط عند تغيّر الحالة //    ;if (dev[name] !== on) dev[name] = on    86
 87  }
```

4 · Conditions and protection

water_ok() is the one place that decides "is there enough water?". Irrigation checks it before and during every second, and checks the pump current: a pump that runs dry or is not connected draws no current, so the program stops it and remembers the failure.

- A level above 100 % is treated as an invalid reading, not as "full".
- The pump is always switched off at the end, whatever stopped the irrigation.

wb/rules.js

```
هل يوجد ماء كافٍ؟ //                       } ()function waterOk  101
قراءة غير صالحة = %100 أكثر من //   ;return waterLevel >= MIN_WATER && waterLevel <= 100    102
103  }
```

```
109  // >>> IRRIGATION
// تُنفّذ كل ثانية أثناء الري             } ()function irrigationSecond  110
// الماء نقص أثناء الري                       } if (!waterOk())    111
// نخبر بالسبب  ;log("IRRIGATION stopped: LOW WATER")      112
// المضخة تعمل ولا يمر تيار   } else if (dev[PUMP] && pumpAmps < MIN_PUMP_AMPS) {    113
// نتذكر العطل (يظهر الإنذار)                    ;pumpFailed = true      114
// رسالة العطل       ;log("PUMP FAILURE: no current")      115
// ما زال هناك وقت         } else if (irrigationLeft > 0) {    116
// نُبقي المضخة تعمل                  ;setRelay(PUMP, true)      117
// ثانية أقل   ;irrigationLeft = irrigationLeft - 1      118
// نكمل في الثانية القادمة                              ;return      119
120    }
// نهاية الري (عادية أو بسبب الحماية                  ;setRelay(PUMP, false)    121
// لم يعد هناك ري                      ;irrigating = false    122
// رسالة نهاية الري                   ;log("IRRIGATION end")    123
124  }
125  // <<< IRRIGATION
```

5 · AUTO mode

Every second: the fan follows the temperature with two thresholds (ON above 30 °C, OFF below 27 °C — between them it keeps its state, so it does not click on and off), the light follows the clock (06:00–20:00), and irrigation starts once at 07:00, 12:00 and 17:00 if there is water.

- No temperature reading → the fan runs (the safe side for the plants).
- last_irrigation_hour prevents watering again and again during the same hour.

wb/rules.js

```
// الوضع التلقائي                   } ()function autoControl  171
// الساعة الحالية (0 إلى 23)         ;()var hour = new Date().getHours    172
173    // >>> FAN
// لا قراءة حرارة: نشغّل المروحة للأمان              } if (temperature === null)    174
// تشغيل المروحة                     ;setRelay(FAN, true)      175
// حار جداً      } else if (temperature > MAX_TEMP) {    176
// تشغيل المروحة                     ;setRelay(FAN, true)      177
// برد بما يكفي  } else if (temperature < FAN_OFF_TEMP) {    178
// إيقاف المروحة (بين 27 و 30 تبقى كما هي                  ;setRelay(FAN, false)      179
180    }
181    // <<< FAN
182    // >>> LIGHT
// داخل ساعات الإضاءة   } if (hour >= LIGHT_ON_HOUR && hour < LIGHT_OFF_HOUR)    183
// تشغيل الإضاءة                  ;setRelay(LIGHT, true)      184
// خارج ساعات الإضاءة                                } else {    185
// إطفاء الإضاءة                ;setRelay(LIGHT, false)      186
187    }
188    // <<< LIGHT
// حان وقت ري جديد   } if (IRRIGATION_HOURS.indexOf(hour) >= 0 && hour !== lastIrrigationHour)    189
// نتذكر الساعة حتى لا نكرر الري              ;lastIrrigationHour = hour      190
// يوجد ماء؟                       } if (waterOk())      191
// رسالة بداية الري              ;log("IRRIGATION start")        192
كل ثانية irrigationSecond ستستدعي ()irrigating = true;                    // control        193
ثوانٍ irrigationLeft = PUMP_SECONDS;        // 5        194
// لا يوجد ماء                                  } else {      195
// نتخطى الري // ;log("IRRIGATION skipped: LOW WATER")        196
197      }
198    }
199  }
```

6 · DOSING mode

Check the water, switch B off, run A for (dose ÷ flow) seconds, switch A off, run B, switch B off, print DOSING DONE. A and B are never on together. The volumes are only as accurate as the flow calibration of the real pumps (here 2 ml/s).

- Started by the pushbutton farm/start_dosing (MQTT tab); setTimeout chains the three steps.
- The Smart Farm counts the millilitres really pumped, so you can check the doses.

wb/rules.js

```
127  // >>> DOSING
// الخطوة 1: المضخة A وحدها                  } ()function doseAStart  128
// تأكد أن B متوقفة قبل تشغيل A                ;setRelay(DOSE_B, false)    129
```

```
A تشغيل //                  ;setRelay(DOSE_A, true)    130
A رسالة الجرعة //         ;log("DOSING A {} ml", DOSE_A_ML)    131
ثوانٍ 5 = (2 ÷ 10) بعد //   ;setTimeout(doseBStart, DOSE_A_ML / FLOW_ML_PER_SEC * 1000)    132
133  }
134
B ثم نشغّل A نطفئ :2 الخطوة //                    } ()function doseBStart  135
A إيقاف //                  ;setRelay(DOSE_A, false)    136
B ثم تشغيل //                   ;setRelay(DOSE_B, true)    137
B رسالة الجرعة //         ;log("DOSING B {} ml", DOSE_B_ML)    138
ثانية 2.5 = (2 ÷ 5) بعد //   ;setTimeout(dosingDone, DOSE_B_ML / FLOW_ML_PER_SEC * 1000)    139
140  }
141
النهاية :3 الخطوة //                    } ()function dosingDone  142
B إيقاف //                  ;setRelay(DOSE_B, false)    143
العودة إلى التلقائي //                          ;"mode = "AUTO    144
انتهت الجرعات //                       ;log("DOSING DONE")    145
146  }
147  // <<< DOSING
```

7 · TEST mode

Each output runs for 2 s in turn: fan, light, dose A, dose B, then the pump — and the pump only if there is water. The protection stays active in test mode.

- Use it after wiring a real farm: every relay should click once and every load should run.
- Dose A and dose B run one after the other, never together.

wb/rules.js

```
149  var TEST_DEVICES = [["FAN", FAN], ["LIGHT", LIGHT], ["DOSE A", DOSE_A], ["DOSE B", DOSE_B], ["PUMP",
↪ ترتيب الاختبار //   ;[[PUMP
150
بالترتيب TEST_SECONDS جهاز واحد كل //                      } ()function testNext  151
نطفئ الجهاز السابق //   ;if (testStep > 0) setRelay(TEST_DEVICES[testStep - 1][1], false)    152
انتهت القائمة //    } if (testStep >= TEST_DEVICES.length)    153
العودة إلى التلقائي //                          ;"mode = "AUTO      154
انتهى الاختبار //                      ;log("TEST DONE")      155
لا خطوة أخرى //                                ;return      156
157    }
اسم الجهاز //     ;var name = TEST_DEVICES[testStep][0]    158
مرحّل الجهاز //    ;var relay = TEST_DEVICES[testStep][1]    159
الخطوة التالية //                  ;testStep = testStep + 1    160
الحماية لا تُلغى في الاختبار //       } if (relay === PUMP && !waterOk())    161
لا نشغّل المضخة فارغة //    ;log("TEST PUMP skipped: LOW WATER")      162
ننتقل مباشرة إلى النهاية //                              ;()testNext      163
لا نشغّل شيئاً هنا //                                ;return      164
165    }
اسم الجهاز الذي نختبره //                     ;log("TEST {}", name)    166
تشغيله //                    ;setRelay(relay, true)    167
الجهاز التالي: بعد ثانيتين: // ;setTimeout(testNext, TEST_SECONDS * 1000)    168
169  }
```

8 · Alarms

One if/elif chain picks the most important alarm: SENSOR ERROR, LOW WATER, PUMP FAILURE, HIGH TEMPERATURE, OUT OF RANGE, or NONE. The alarm lamp is on for any alarm, and the alarm is printed only when it changes.

- The order of the elif lines is the priority: a missing sensor matters more than a high temperature.
- PUMP FAILURE stays until the program restarts (the pump must be checked by a person).

wb/rules.js

```
201  // >>> ALARMS
اختيار أهم إنذار وتشغيل المصباح //                    } ()function checkAlarms  202
نحفظ الإنذار السابق //                        ;var old = alarm    203
قراءة غير //   } if (temperature === null || co2 === null || ph < 1 || ph > 14 || waterLevel > 100)    204
↪ صالحة
خطأ حساس //                 ;"alarm = "SENSOR ERROR      205
الماء قليل //                  } else if (!waterOk()) {    206
نقص الماء //                    ;"alarm = "LOW WATER      207
المضخة لم تسحب تياراً //                  } else if (pumpFailed) {    208
عطل المضخة //                 ;"alarm = "PUMP FAILURE      209
```

```
10 حرارة عالية //      } else if (temperature > MAX_TEMP) {    210
11 ارتفاع الحرارة //             ;"alarm = "HIGH TEMPERATURE      211
12 خارج الحدود //   } else if (ph < PH_MIN || ph > PH_MAX || ec < EC_MIN || ec > EC_MAX || co2 > CO2_MAX) {    212
13 قيمة خارج المجال //                 ;"alarm = "OUT OF RANGE      213
14 كل شيء طبيعي //                                 } else {    214
15 لا يوجد إنذار //                           ;"alarm = "NONE      215
16 216    }
17 المصباح يعمل عند وجود أي إنذار //   ;setRelay(ALARM_LAMP, alarm !== "NONE")    217
18 نسجّل الإنذار عند تغيّره فقط //   ;if (alarm !== old) log("ALARM {}", alarm)    218
19 219  }
20 220  // <<< ALARMS
```

9 · Display and monitoring

Every 5 s a STATUS line goes to the wb-rules log and the values to the virtual device "farm" (MQTT tab). Its two pushbuttons start DOSING and TEST. setInterval(control, 1000) is the main loop.

- STATUS mode=… temp=… … alarm=… day=… is the same line on the three controllers.
- The status panel of this page reads that line from the running program.

wb/rules.js

```
1 farm :اسم الجهاز //                     } ,"defineVirtualDevice("farm  41
2 العنوان الظاهر //                           ,"title: "Smart Farm    42
3 (Controls) عناصر الجهاز //                                          } :cells    43
4 الوضع الحالي //        ,mode: { type: "text", value: "AUTO", readonly: true }      44
5 زر: تشغيل الجرعات //          ,start_dosing: { type: "pushbutton" }      45
6 زر: وضع الاختبار //           ,start_test: { type: "pushbutton" }      46
7 (-1 = لا قراءة) الحرارة //     ,temperature: { type: "value", value: 0, readonly: true }      47
8 الرطوبة //        ,humidity: { type: "value", value: 0, readonly: true }      48
 49      co2: { type: "value", value: 0, readonly: true },             // CO2
 50      ph: { type: "value", value: 0, readonly: true },              // pH
 51      ec: { type: "value", value: 0, readonly: true },              // EC
% مستوى الماء //     ,water_level: { type: "value", value: 0, readonly: true }      52
الإنذار //       ,alarm: { type: "text", value: "NONE", readonly: true }      53
يوم النمو //              day: { type: "value", value: 1, readonly: true }      54
نهاية العناصر //                                                  {    55
نهاية تعريف الجهاز //                                                  ;({  56
farm الحالة في السجل وفي الجهاز الافتراضي //                   } ()function showStatus  227
228    log("STATUS mode={} temp={} hum={} co2={} ph={} ec={} level={} pump={} fan={} light={} alarm={} day={}",
↪ Python نفس سطر //
229      mode, temperature, humidity, co2, ph, ec, waterLevel, onOff(PUMP), onOff(FAN), onOff(LIGHT), alarm,
↪ القيم //   ;(()growthDay
MQTT تحديث لوحة الحالة في //                 ;dev["farm/mode"] = mode    230
لا قراءة = dev["farm/temperature"] = temperature === null ? -1 : temperature;   // -1    231
لا قراءة = dev["farm/humidity"] = humidity === null ? -1 : humidity;   // -1    232
لا قراءة = dev["farm/co2"] = co2 === null ? -1 : co2;   // -1    233
234    dev["farm/ph"] = ph;                     // pH
235    dev["farm/ec"] = ec;                     // EC
مستوى الماء //     ;dev["farm/water_level"] = waterLevel    236
الإنذار //               ;dev["farm/alarm"] = alarm    237
يوم النمو //            ;()dev["farm/day"] = growthDay    238
239  }
---------- الأزرار: قواعد تبدأ الأوضاع (4 ---------- //  241
قاعدة زر الجرعات //             } ,"defineRule("dosing_button  242
(MQTT) عند الضغط على زر الجرعات //         ,"whenChanged: "farm/start_dosing    243
ماذا نفعل عند الضغط //                       } () then: function    244
وضع آخر أو ري يعمل الآن //   ;if (mode !== "AUTO" || irrigating) return      245
لا تسميد بدون ماء //                      } if (!waterOk())      246
نخبر بالسبب //   ;log("DOSING cancelled: LOW WATER")        247
لا نبدأ //                               ;return        248
249      }
وضع الجرعات //                        ;"mode = "DOSING      250
1 الخطوة //                           ;()doseAStart      251
252    }
253  });
254
قاعدة زر الاختبار //                } ,"defineRule("test_button  255
(MQTT) عند الضغط على زر الاختبار //           ,"whenChanged: "farm/start_test    256
ماذا نفعل عند الضغط //                       } () then: function    257
وضع آخر أو ري يعمل الآن //   ;if (mode !== "AUTO" || irrigating) return      258
وضع الاختبار //                           ;"mode = "TEST      259
من أول جهاز //                             ;testStep = 0      260
الجهاز الأول //                              ;()testNext      261
262    }
263  });
```

```
264
---------- كل ثانية ()control مؤقّت يستدعي: "الحلقة الرئيسية" (5 ---------- //  265
} ()function control  266
// تُنفّذ كل ثانية                        قراءة الحساسات (readSensors();                              // 1    267
الري الجاري له الأولوية (if (irrigating) {                                  // 2    268
// ثانية واحدة من الري                    ;()irrigationSecond      269
تعمل الخطوات بمؤقّتاتها TEST و DOSING في (else if (mode === "AUTO") {             // 3 {    270
// المروحة والإضاءة والري                          ;()autoControl      271
272    }
الإنذارات (checkAlarms();                             // 4    273
الحالة كل 5 ثوانٍ (if (loop % 5 === 0) showStatus();         // 5    274
// الدورة التالية                          ;loop = loop + 1    275
276  }
277
wb-rules رسالة البداية في سجل //   ;log("Smart farm started (Wiren Board 7.4)")  278
// تكرار control كل 1000 ميلي ثانية                ;setInterval(control, 1000)  279
```

6. Complete program

wb/rules.js

```
// COMPETITION MASTER 03 — Wiren Board 7.4 Complete Smart Farm (wb-rules)
// نفس منطق مشروعي Raspberry Pi و ESP32 — هنا نقرأ ونكتب "Controls" عبر MQTT بدلاً من الأرجل

// ---------- 1) الإعدادات: نفس القيم في المشاريع الثلاثة ----------
var MAX_TEMP = 30;            // فوق 30 درجة: تشغيل المروحة
var FAN_OFF_TEMP = 27;        // تحت 27 درجة: إيقاف المروحة
var MIN_WATER = 20;           // أقل مستوى ماء مسموح (%) لتشغيل المضخة
var PUMP_SECONDS = 5;         // مدة الري بالثواني
var IRRIGATION_HOURS = [7, 12, 17]; // ساعات الري في اليوم
var LIGHT_ON_HOUR = 6;        // ساعة تشغيل الإضاءة
var LIGHT_OFF_HOUR = 20;      // ساعة إطفاء الإضاءة
var DOSE_A_ML = 10;           // جرعة السماد A بالمليلتر
var DOSE_B_ML = 5;            // جرعة السماد B بالمليلتر
var FLOW_ML_PER_SEC = 2;      // تدفق مضخة الجرعات (يحتاج معايرة على الجهاز الحقيقي)
var PH_MIN = 5.5;             // أقل pH مقبول
var PH_MAX = 6.5;             // أعلى pH مقبول
var EC_MIN = 1.0;             // أقل EC مقبول (mS/cm)
var EC_MAX = 2.5;             // أعلى EC مقبول
var CO2_MAX = 1500;           // أعلى CO2 مقبول (ppm)
var MIN_PUMP_AMPS = 0.2;      // أقل تيار يثبت أن المضخة تعمل فعلاً (أمبير)
var TEST_SECONDS = 2;         // مدة تشغيل كل جهاز في وضع الاختبار

// >>> PINS
// ---------- 2) التوصيلات = أسماء الـControls بصيغة "جهاز/عنصر" ----------
var TEMP = "air-sensor_2/Temperature";  // حساس الهواء RS-485 (عنوان 2 Modbus): الحرارة
var HUM = "air-sensor_2/Humidity";      // نفس الحساس: الرطوبة
var CO2 = "air-sensor_2/CO2";           // نفس الحساس: CO2
var PH_VOLTS = "wb-adc/A1";             // جهد حساس pH على المدخل A1
var EC_VOLTS = "wb-adc/A2";             // جهد حساس EC على A2
var LEVEL_VOLTS = "wb-adc/A3";          // جهد حساس المستوى على A3
var CURRENT_VOLTS = "wb-adc/A4";        // جهد حساس التيار ACS712 على A4
var PUMP = "wb-mr6c_32/K1";             // وحدة المرحّلات WB-MR6C (عنوان 32): K1 مضخة الري
var DOSE_A = "wb-mr6c_32/K2";           // K2 مضخة الجرعات A
var DOSE_B = "wb-mr6c_32/K3";           // K3 مضخة الجرعات B
var FAN = "wb-mr6c_32/K4";              // K4 المروحة
var LIGHT = "wb-mr6c_32/K5";            // K5 الإضاءة
var ALARM_LAMP = "wb-mr6c_32/K6";       // K6 مصباح الإنذار
// <<< PINS

// لوحة الحالة: جهاز افتراضي يظهر في تبويب MQTT (وفي واجهة Wiren Board على الجهاز الحقيقي)
defineVirtualDevice("farm", {
    title: "Smart Farm",                // العنوان الظاهر
    cells: {                            // عناصر الجهاز (Controls)
        mode: { type: "text", value: "AUTO", readonly: true },   // الوضع الحالي
        start_dosing: { type: "pushbutton" },                    // زر: تشغيل الجرعات
        start_test: { type: "pushbutton" },                      // زر: وضع الاختبار
        temperature: { type: "value", value: 0, readonly: true }, // الحرارة (-1 = لا قراءة)
        humidity: { type: "value", value: 0, readonly: true },   // الرطوبة
        co2: { type: "value", value: 0, readonly: true },        // CO2
        ph: { type: "value", value: 0, readonly: true },         // pH
        ec: { type: "value", value: 0, readonly: true },         // EC
        water_level: { type: "value", value: 0, readonly: true }, // مستوى الماء %
        alarm: { type: "text", value: "NONE", readonly: true },  // الإنذار
        day: { type: "value", value: 1, readonly: true }         // يوم النمو
    }                                   // نهاية العناصر
});                                     // نهاية تعريف الجهاز

// ---------- 3) حالة النظام ----------
var mode = "AUTO";            // الوضع الحالي: AUTO أو DOSING أو TEST
var alarm = "NONE";           // الإنذار الحالي
var pumpFailed = false;       // يصبح true إذا شغّلنا المضخة ولم يمر تيار
var temperature = null;       // آخر قراءة حرارة (null = لا قراءة)
var humidity = null;          // آخر قراءة رطوبة
var co2 = null;               // آخر قراءة CO2
var ph = 0;                   // آخر قراءة pH
var ec = 0;                   // آخر قراءة EC
var waterLevel = 0;           // مستوى الماء %
var pumpAmps = 0;             // تيار المضخة بالأمبير
var lastIrrigationHour = -1;  // آخر ساعة تم فيها الري (حتى لا نكرره)
var startTime = Date.now();   // وقت بدء التشغيل لحساب يوم النمو (ميلي ثانية)
var irrigating = false;       // هل الري جارٍ الآن؟
```

```
var irrigationLeft = 0;       // ثوانٍ متبقية من الري
var testStep = 0;             // رقم الجهاز التالي في الاختبار
var loop = 0;                 // عدّاد الدورات

function round2(x) {          // تقريب إلى رقمين بعد الفاصلة
  return Math.round(x * 100) / 100;       // مثال: 6.1987 → 6.2
}

function sensorValue(name) {              // قيمة حساس Modbus، أو null إذا لم يرد
  if (getControl(name).getError() !== "") return null;   // wb-mqtt-serial يضع خطأ عند انقطاع الحساس
  return dev[name];                       // القيمة الأخيرة المنشورة عبر MQTT
}

function setRelay(name, on) {             // تشغيل أو إيقاف مرحّل
  //if (dev[name] !== on) dev[name] = on;    // نكتب فقط عند تغيّر الحالة (أمر Modbus واحد)
}

// >>> SENSORS
function readSensors() {                  // قراءة كل الحساسات
  temperature = sensorValue(TEMP);        // الحرارة °C (أو null)
  humidity = sensorValue(HUM);            // الرطوبة % (أو null)
  co2 = sensorValue(CO2);                 // CO2 ppm (أو null)
  ph = round2(dev[PH_VOLTS] * 3.5);       // pH = 3.5 × الجهد // نفس معادلات Python: ec = round2(dev[EC_VOLTS] / 0.5);         // EC = الجهد ÷ 0.5
  waterLevel = Math.round(dev[LEVEL_VOLTS] / 3.0 * 100);   // 3 فولت = 100%
  pumpAmps = round2((dev[CURRENT_VOLTS] - 2.5) / 0.185);   // ACS712: 2.5 فولت = صفر أمبير
}
// <<< SENSORS

function waterOk() {                      // هل يوجد ماء كافٍ؟
  return waterLevel >= MIN_WATER && waterLevel <= 100;   // قراءة غير صالحة = أكثر من 100%
}

function growthDay() {                    // يوم النمو منذ بدء التشغيل
  return Math.floor((Date.now() - startTime) / 86400000) + 1;   // 86400000 ميلي ثانية = يوم
}

// >>> IRRIGATION
function irrigationSecond() {             // تُنفّذ كل ثانية أثناء الري
  if (!waterOk()) {                       // الماء نقص أثناء الري
    log("IRRIGATION stopped: LOW WATER");  // نخبر بالسبب
  } else if (dev[PUMP] && pumpAmps < MIN_PUMP_AMPS) {   // المضخة تعمل ولا يمر تيار
    pumpFailed = true;                    // نتذكر العطل (يظهر الإنذار)
    log("PUMP FAILURE: no current");      // رسالة العطل
  } else if (irrigationLeft > 0) {        // ما زال هناك وقت
    setRelay(PUMP, true);                 // نُبقي المضخة تعمل
    irrigationLeft = irrigationLeft - 1;  // ثانية أقل
    return;                               // نكمل في الثانية القادمة
  }
  setRelay(PUMP, false);                  // نهاية الري (عادية أو بسبب الحماية)
  irrigating = false;                     // لم يعد هناك ري
  log("IRRIGATION end");                  // رسالة نهاية الري
}
// <<< IRRIGATION

// >>> DOSING
function doseAStart() {                   // الخطوة 1: المضخة A وحدها
  setRelay(DOSE_B, false);                // تأكد أن B متوقفة قبل تشغيل A
  setRelay(DOSE_A, true);                 // تشغيل A
  log("DOSING A {} ml", DOSE_A_ML);       // رسالة الجرعة A
  setTimeout(doseBStart, DOSE_A_ML / FLOW_ML_PER_SEC * 1000);   // بعد (10 ÷ 2) = 5 ثوانٍ
}

function doseBStart() {                   // الخطوة 2: نطفئ A ثم نشغّل B
  setRelay(DOSE_A, false);                // إيقاف A
  setRelay(DOSE_B, true);                 // ثم تشغيل B
  log("DOSING B {} ml", DOSE_B_ML);       // رسالة الجرعة B
  setTimeout(dosingDone, DOSE_B_ML / FLOW_ML_PER_SEC * 1000);   // بعد (5 ÷ 2) = 2.5 ثانية
}

function dosingDone() {                   // الخطوة 3: النهاية
  setRelay(DOSE_B, false);                // إيقاف B
  mode = "AUTO";                          // العودة إلى التلقائي
  log("DOSING DONE");                     // انتهت الجرعات
}
// <<< DOSING
```

```
var TEST_DEVICES = [["FAN", FAN], ["LIGHT", LIGHT], ["DOSE A", DOSE_A], ["DOSE B", DOSE_B], ["PUMP", PUMP]]; // ترتيب الاختبار

// جهاز واحد كل TEST_SECONDS بالترتيب                     } ()function testNext
//   ;if (testStep > 0) setRelay(TEST_DEVICES[testStep - 1][1], false) نطفئ الجهاز السابق
//    } if (testStep >= TEST_DEVICES.length) انتهت القائمة
// العودة إلى التلقائي                          ;"mode = "AUTO
// انتهى الاختبار                       ;log("TEST DONE")
// لا خطوة أخرى                                ;return
  }
//     ;var name = TEST_DEVICES[testStep][0] اسم الجهاز
//    ;var relay = TEST_DEVICES[testStep][1] مرحّل الجهاز
// الخطوة التالية                  ;testStep = testStep + 1
// الحماية لا تُلغى في الاختبار       } if (relay === PUMP && !waterOk())
//    ;log("TEST PUMP skipped: LOW WATER") لا نشغّل المضخة فارغة
// ننتقل مباشرة إلى النهاية                          ;()testNext
// لا نشغّل شيئاً هنا                               ;return
  }
// اسم الجهاز الذي نختبره                    ;log("TEST {}", name)
// تشغيله                    ;setRelay(relay, true)
// بعد ثانيتين: الجهاز التالي ;setTimeout(testNext, TEST_SECONDS * 1000)
}

// الوضع التلقائي                    } ()function autoControl
// الساعة الحالية (0 إلى 23)         ;()var hour = new Date().getHours
  // >>> FAN
// لا قراءة حرارة: نشغّل المروحة للأمان               } if (temperature === null)
// تشغيل المروحة                     ;setRelay(FAN, true)
//      } else if (temperature > MAX_TEMP) { حار جداً
// تشغيل المروحة                     ;setRelay(FAN, true)
//  } else if (temperature < FAN_OFF_TEMP) { برد بما يكفي
// إيقاف المروحة (بين 27 و 30 تبقى كما هي)                   ;setRelay(FAN, false)
  }
  // <<< FAN
  // >>> LIGHT
//   } if (hour >= LIGHT_ON_HOUR && hour < LIGHT_OFF_HOUR) داخل ساعات الإضاءة
// تشغيل الإضاءة                   ;setRelay(LIGHT, true)
// خارج ساعات الإضاءة                                } else {
// إطفاء الإضاءة                 ;setRelay(LIGHT, false)
  }
  // <<< LIGHT
//   } if (IRRIGATION_HOURS.indexOf(hour) >= 0 && hour !== lastIrrigationHour) حان وقت ري جديد
// نتذكر الساعة حتى لا نكرر الري              ;lastIrrigationHour = hour
// يوجد ماء؟                        } if (waterOk())
// رسالة بداية الري              ;log("IRRIGATION start")
كل ثانية irrigationSecond ستستدعي ()irrigating = true;                    // control
ثوانٍ irrigationLeft = PUMP_SECONDS;        // 5
// لا يوجد ماء                                } else {
// نتخطى الري // ;log("IRRIGATION skipped: LOW WATER")
    }
  }
}

// >>> ALARMS
// اختيار أهم إنذار وتشغيل المصباح                    } ()function checkAlarms
// نحفظ الإنذار السابق                          ;var old = alarm
//   } if (temperature === null || co2 === null || ph < 1 || ph > 14 || waterLevel > 100) قراءة غير صالحة
// خطأ حساس                 ;"alarm = "SENSOR ERROR
// الماء قليل                   } else if (!waterOk()) {
// نقص الماء                     ;"alarm = "LOW WATER
// المضخة لم تسحب تياراً                  } else if (pumpFailed) {
// عطل المضخة                 ;"alarm = "PUMP FAILURE
// حرارة عالية      } else if (temperature > MAX_TEMP) {
// ارتفاع الحرارة             ;"alarm = "HIGH TEMPERATURE
//   } else if (ph < PH_MIN || ph > PH_MAX || ec < EC_MIN || ec > EC_MAX || co2 > CO2_MAX) { خارج الحدود
// قيمة خارج المجال                ;"alarm = "OUT OF RANGE
// كل شيء طبيعي                                } else {
// لا يوجد إنذار                         ;"alarm = "NONE
  }
//   ;setRelay(ALARM_LAMP, alarm !== "NONE") المصباح يعمل عند وجود أي إنذار
//   ;if (alarm !== old) log("ALARM {}", alarm) نسجّل الإنذار عند تغيّره فقط
}
// <<< ALARMS

"OFF" أو "function onOff(name) {                    // "ON لعرض حالة مرحّل
// المرحّل مغلق = الجهاز يعمل               ;"if (dev[name]) return "ON
// متوقف                             ;"return "OFF
```

```
}

farm الافتراضي الجهاز في وفي السجل في الحالة //                    } ()function showStatus
  log("STATUS mode={} temp={} hum={} co2={} ph={} ec={} level={} pump={} fan={} light={} alarm={} day={}",   //
↪ Python نفس سطر
    mode, temperature, humidity, co2, ph, ec, waterLevel, onOff(PUMP), onOff(FAN), onOff(LIGHT), alarm,
↪ القيم //   ;(()growthDay
MQTT في الحالة لوحة تحديث //                  ;dev["farm/mode"] = mode
قراءة لا = dev["farm/temperature"] = temperature === null ? -1 : temperature;   // -1
قراءة لا = dev["farm/humidity"] = humidity === null ? -1 : humidity;   // -1
قراءة لا = dev["farm/co2"] = co2 === null ? -1 : co2;   // -1
  dev["farm/ph"] = ph;                      // pH
  dev["farm/ec"] = ec;                      // EC
ماء مستوى //     ;dev["farm/water_level"] = waterLevel
الإنذار //               ;dev["farm/alarm"] = alarm
نمو يوم //            ;()dev["farm/day"] = growthDay
}

---------- الأزرار: قواعد تبدأ الأوضاع (4 ---------- //
جرعات زر قاعدة //              } ,"defineRule("dosing_button
(MQTT) جرعات زر على الضغط عند //         ,"whenChanged: "farm/start_dosing
الضغط عند نفعل ماذا //                      } () then: function
الآن يعمل ري أو آخر وضع //   ;if (mode !== "AUTO" || irrigating) return
ماء بدون تسميد لا //                       } if (!waterOk())
بالسبب نخبر //   ;log("DOSING cancelled: LOW WATER")
نبدأ لا //                                ;return
    }
الجرعات وضع //                         ;"mode = "DOSING
الخطوة 1 //                            ;()doseAStart
  }
});

الاختبار زر قاعدة //                } ,"defineRule("test_button
(MQTT) الاختبار زر على الضغط عند //           ,"whenChanged: "farm/start_test
الضغط عند نفعل ماذا //                       } () then: function
الآن يعمل ري أو آخر وضع //   ;if (mode !== "AUTO" || irrigating) return
الاختبار وضع //                             ;"mode = "TEST
جهاز أول من //                               ;testStep = 0
الأول الجهاز //                                ;()testNext
  }
});

---------- ثانية كل ()control يستدعي مؤقّت :"الحلقة الرئيسية" (5 ---------- //
ثانية كل تُنفّذ //                         } ()function control
الحساسات قراءة (readSensors();                                // 1
الأولوية له الجاري الري (if (irrigating) {                          // 2
الري من واحدة ثانية //                       ;()irrigationSecond
بمؤقّتاتها تعمل الخطوات و TEST في DOSING (else if (mode === "AUTO") {             // 3 {
والري والإضاءة المروحة //                              ;()autoControl
  }
الإنذارات (checkAlarms();                                 // 4
ثوانٍ 5 كل الحالة (if (loop % 5 === 0) showStatus();         // 5
التالية الدورة //                              ;loop = loop + 1
}

سجل في rules-wb رسالة البداية //   ;log("Smart farm started (Wiren Board 7.4)")
ثانية 1000 كل control تكرار //                 ;setInterval(control, 1000)

```

7. Test scenarios and results

Results of the scenarios run in the browser on 2026-11-05 (real runtime of the simulator).

Scenario	What it does	Expected	Result
1 · Normal	08:00, 25 °C, water 75 %: run for 12 s without touching anything.	STATUS lines with all readings, mode AUTO, light ON (06–20 h), fan and pump OFF, alarm NONE, alarm lamp off.	PASS
2 · High temperature (25 → 35 °C)	At 3 s the air goes to 35 °C, at 9 s to 28.5 °C (between the two thresholds), at 15 s to 25 °C.	Fan ON and HIGH TEMPERATURE alarm above 30 °C; at 28.5 °C the alarm clears but the fan stays ON (hysteresis); below 27 °C the fan stops.	PASS
3 · Water low	11:59:55 — at 2 s the reservoir drops to 10 %; the 12:00 irrigation comes 3 s later.	LOW WATER alarm, alarm lamp ON, "IRRIGATION skipped: LOW WATER": the pump never runs.	PASS
4 · Irrigation at 12:00	Start at 11:59:56 and run 25 s.	"IRRIGATION start", the pump runs about 5 s (current measured every second), "IRRIGATION end", and only once in that hour.	PASS
5 · Dosing A then B	At 2 s press farm/start_dosing (MQTT).	DOSING A 10 ml (5 s), then DOSING B 5 ml (2.5 s), never together, then DOSING DONE and back to AUTO. Volumes come from the farm model of the pumps (2 ml/s).	PASS
6 · Test mode	21:00 (light off by schedule): at 2 s press farm/start_test (MQTT).	TEST FAN, TEST LIGHT, TEST DOSE A, TEST DOSE B, TEST PUMP (only because the water is OK), TEST DONE; every load runs once, then AUTO switches the light off again.	PASS
7 · Sensor failure	At 3 s the air sensor (RS-485, address 2) gets the fault "Sensor disconnected"; at 10 s the fault is cleared.	SENSOR ERROR alarm, temp=None/null in STATUS, the fan runs (safe side) while the reading is missing; the alarm clears after repair.	PASS
8 · Lighting schedule	Start at 19:59:52 and run 15 s.	The grow light is ON before 20:00 and switches OFF after 20:00 (LIGHT_OFF_HOUR).	PASS
9 · Wiring error (pump wire loose)	The wire from relay contact NO1 to the ACS712 is removed, then the 12:00 irrigation starts (start 11:59:56).	The relay clicks but no current flows: "PUMP FAILURE: no current", PUMP FAILURE alarm, the relay is switched off at once.	PASS
10 · Full system	11:59:52: irrigation at 12:00, at 14 s the air reaches 35 °C, at 18 s press farm/start_dosing (MQTT), at 32 s the air is back to 25 °C.	Irrigation (≈5 s), fan + HIGH TEMPERATURE alarm, dosing A then B, then every alarm clears and the fan stops: the system returns to normal AUTO.	PASS

8. Change One Thing — exercises

1. Change the fan threshold

   Set MAX_TEMP to 28 and FAN_OFF_TEMP to 25. Run, then set the farm air temperature to 29 °C.

   You should see: The fan starts at 29 °C now and the HIGH TEMPERATURE alarm appears.

2. Change the irrigation time

   Set PUMP_SECONDS to 8 and IRRIGATION_HOURS to include the current hour. Run and watch the pump.

   You should see: IRRIGATION start … IRRIGATION end, about 8 s apart; the farm reservoir flow shows the pump running.

3. Move the fan to another relay

   Move the fan wire from K4 NO to K6 NO and the alarm lamp to K4, then swap FAN and ALARM_LAMP in the PINS part.

   You should see: Both work again. If you change only the wire or only the code, the wrong load switches.

4. Disconnect the level sensor

   While the program runs, delete the signal wire of the water-level sensor.

   You should see: The level reads wrong (near 0): LOW WATER alarm, the lamp lights, irrigation and dosing are refused. Put the wire back (Ctrl+Z).

5. Change the dose

   Set DOSE_A_ML to 20 and DOSE_B_ML to 10, then start DOSING.

   You should see: A runs 10 s and B 5 s; the Smart Farm totals show about 20 ml and 10 ml. Changing FLOW_ML_PER_SEC without changing the pump only changes the time, not the real flow — that is why calibration matters.

6. Break the pump while it runs

   Select the main pump, set its fault to "Pump failure" and start irrigation (put the current hour in IRRIGATION_HOURS).

   You should see: The relay closes but the ACS712 measures no current: PUMP FAILURE: no current, alarm PUMP FAILURE.

7. Change the lighting hours

   Set LIGHT_ON_HOUR to 8 and LIGHT_OFF_HOUR to 18. Use the clock (Fast-forward) to pass 18:00.

   You should see: The grow light switches off at 18:00 instead of 20:00.

8. Tighten the pH range

   Set PH_MAX to 6.0 (the farm solution is about pH 6.2).

   You should see: ALARM OUT OF RANGE and the lamp lights; nothing else changes because OUT OF RANGE only warns.

9. Building it on real hardware

1. Set the Modbus addresses: air sensor 2, WB-MR6C 32 (all devices 9600 8N1 on RS-485-1, A↔A, B↔B, 120 Ω termination at the bus end).
2. Add both devices in the web interface: Settings → Configs → wb-mqtt-serial (choose the device templates).
3. Check the control names in Devices: they must match TEMP, HUM, CO2 and K1–K6 in the PINS part (your real sensor template may use other names).
4. Set A1–A4 to voltage measurement, then copy the script to /etc/wb-rules/farm.js (or the Rules editor); wb-rules reloads it automatically.
5. Watch journalctl -u wb-rules -f (or the Rules log) for the STATUS lines.

10. Libraries to install

- wb-rules (pre-installed on the controller): part of the Wiren Board firmware
- wb-mqtt-serial (Modbus devices): pre-installed; configure in Settings → Configs
- wb-mqtt-adc / wb-mqtt-gpio: pre-installed drivers for A1–A4

11. Simulation vs. real device

- wb-rules really runs (QuickJS), with the real MQTT topics; Linux, the web interface and the drivers are reduced models.
- The RS-485 air sensor is a generic Modbus model (input registers 0 temp×10, 1 humidity×10, 2 CO_2); your real sensor has its own register map and template.
- The A1–A4 inputs have a 5 V / 100 kΩ pull-up in the model, so analog readings are slightly higher (pH 6.3 for 6.2) — calibrate on the real device.
- There is no Wiren Board web UI in the simulator: the MQTT tab shows the same devices and controls.

These projects are training references; they do not prove which equipment the final uses nor cover every competition module.